

SpendPartition: Scalable Cross-Delegation Spending Control for Delegated Payments

SC6109 project proposal (Option X, bring your own project) · 30 September 2026
HE HUIZHENGXIONG (G2509011E) · LIU KEFAN (G2509368B) · RAVINJEET SINGH DAVINDAR SINGH (G2509014G) · ZHOU CONGXIANG (G2509183J)

The problem

Wallet owners increasingly hand payment authority to programs that spend without asking for a signature each time: ERC-4337 and ERC-7579 session keys, ERC-7710 delegations, ERC-7715 granted permissions. Most owners hand it to more than one program at once.

Suppose an owner is willing to risk 100 USDC this week across two delegates, and either of them may turn out to need most of it. She grants 80 to each. Every payment then stays inside its own cap while the two together spend 160, and no check on chain objects, because no check is looking at the pair. Splitting the budget 50/50 removes the overshoot and creates a different problem: the first delegate's legitimate request for 80 is refused while the other 50 sits unused. Adjusting the split by hand means the owner has to be online and pay gas every time, which is the work delegation was supposed to remove.

Every deployed control we found counts spending against one delegation, one session, or one account. None counts a set of delegates against the budget the owner actually has in mind.

What already exists

System	What its counter covers
MetaMask Delegation Toolkit, 28 caveat enforcers (amount, period transfer, streaming, allowlist, time, call count)	one delegation
ERC-7579 SmartSessions policies (ERC20SpendingLimit, ValueLimit, UsageLimit, TimeFrame)	one session; state is keyed by session id, token and sender
ERC-7579 SpendingLimitHook, experimental	one account and one token, over a fixed one-week period, and only for ERC-20 transfer calls
AIP (arXiv 2603.24775), CapLease (arXiv 2608.01710)	one authorization instance, enforced off chain

We read the last two rows in source rather than in documentation, because the scope of a counter is where these systems tend to be described loosely. AIP says in §3.4 that aggregate spend enforcement belongs to the runtime and not to the token, which is the same gap seen from the other side.

We are not claiming aggregate budgeting as an idea. Reserving a floor for each user and sharing the remainder is the classical shared-buffer allocation problem, studied since Kamoun and Kleinrock in 1980. What we could not find, as of September 2026, is a contract that applies it to delegated payments and reports what the choice costs.

What we build

One parameterised Solidity contract. Delegate i holds a reserved allocation r_i , the remainder $S = B_G - \sum r_i$ is a shared surplus, and a single parameter $\rho = \sum r_i / B_G$ sets the split. At $\rho = 1$ the contract behaves like today's per-delegation caps. At $\rho = 0$ it is one account-wide pool. In between, each delegate keeps a floor it cannot be denied and competes for the rest. A payment draws from the delegate's own reservation first and then from the surplus, inside the payment path, and is either accepted in full or reverted. Budget windows reset lazily by epoch tag, so no keeper is needed and no loop runs over the delegate set.

Deliberately out of scope: MPC custody, multiple chains, ZK or FHE, a merchant network, dispute handling, and lending unspent allocation with recall. The last one is a result rather than an omission: once value has left the contract it cannot be taken back.

How we will judge it

Safety is the sum of spending in any window staying within B_G under every schedule, checked by property, invariant and fuzz tests and against a second implementation. Isolation is a delegate never being refused a payment that fits inside its own unspent reservation, whatever the others did. Utilisation and fairness are served demand and refusals as ρ moves from 0 to 1 under honest, aggressive and adversarially ordered workloads; we report the curves and do not assume their shape. Cost is gas per payment and at deployment as the delegate count grows.

Where it stands

The contract and its tests are running and public at github.com/liukefan821/spendpartition, and everything below can be reproduced with two commands.

Fifty-three tests pass: the traces we computed by hand in the test plan, fuzzing, and invariant campaigns of 8,192 calls each over six configurations. A second implementation of the same rules, written plainly instead of for cost, is driven through the same random call sequences as the first, and the two are compared value by value after every call. We also broke the contract four times on purpose, one place at a time, and recorded which test catches which break: three of the four are caught by exactly one test each, so we know those three are carrying weight. Tokens that re-enter during the transfer, revert, or return false have a suite of their own.

That second implementation doubles as the cost baseline. A payment through it costs 93,972 gas with ten delegates and 193,452 with fifty, because it scans the delegate list; the packed version stays flat.

Gas figures come from receipts on a local chain, one transaction per payment. A payment costs 47,123 gas whether the contract holds 2 delegates or 50. Deployment grows by about 24,971 gas per delegate, so the part that depends on N is paid once at setup. Sixteen payments issued inside one transaction cost 11,885 each. A single self-contained page, [results/dashboard.html](#), replays six scenes and the ρ sweep from this data: caps that overshoot, a static split that refuses a serviceable request, a shared pool that serves it, a first mover that empties the pool, a reservation that holds the floor, and a slider over ρ .

Why not one of the six options

Options 1 to 6 all sit at the infrastructure layer: an intent execution network, a payment appchain, data availability and rollout benchmarking, a parallel EVM executor, a cross-rollup router. This project sits at the authorization layer instead, and the scalability question it asks is not transactions per second but how gas, storage and refusals behave as the delegate count grows and as an adversary controls the order of arrivals. We picked it because that question has an answer we can measure, and because nothing deployed answers it today.

Team and cost

Four workstreams run in parallel because the interface and the test plan were frozen before implementation: contracts and interfaces; verification; adversarial workloads and the demo; measurement, dashboard, report and video. The project costs nothing to run, needing only Foundry and a local chain, and a public testnet deployment is optional.